

Mobia Checkout Flow – Developer Specification

Sep 30, 2026 · @Moon

1. Overview and problem

The checkout page must be filled from the signed-in customer's real profile, or left empty for guests, never from hardcoded values. Today every order opens with the same dummy details, whoever is buying.

Current behaviour (bug): after any item is added to the cart, Shipping & Contact Details always shows:

Field	Hardcoded value shown today
Full Name	Mobixa Admin
Phone Number	+94 77 123 4567
Email Address	admin@mobilehub.com
Address Line 1	No. 88, Tech Avenue
Address Line 2	Suite 4B
City / Town	Colombo
District	Colombo
Delivery Instructions	Please call before delivery.

Why it matters: orders can be placed with the wrong name, phone and address; couriers deliver to a fake address; customer data in the ERP becomes unreliable; and the admin account's details leak into every customer's screen.

Fix in one line: remove all hardcoded defaults from the checkout form and load prefill values only from the authenticated user's saved profile and addresses via the API.

2. Goals and scope

This spec covers everything from "Add to cart" to the order confirmation screen.

In scope

- Cart page (review items, change quantity, remove items)
- Checkout step 1: Shipping & Contact Details (courier delivery or branch pickup)
- Checkout step 2: Payment method selection
- Order summary, promo codes, totals
- Order placement, stock handling and confirmation page
- Guest checkout and logged-in checkout, including saved addresses

Out of scope

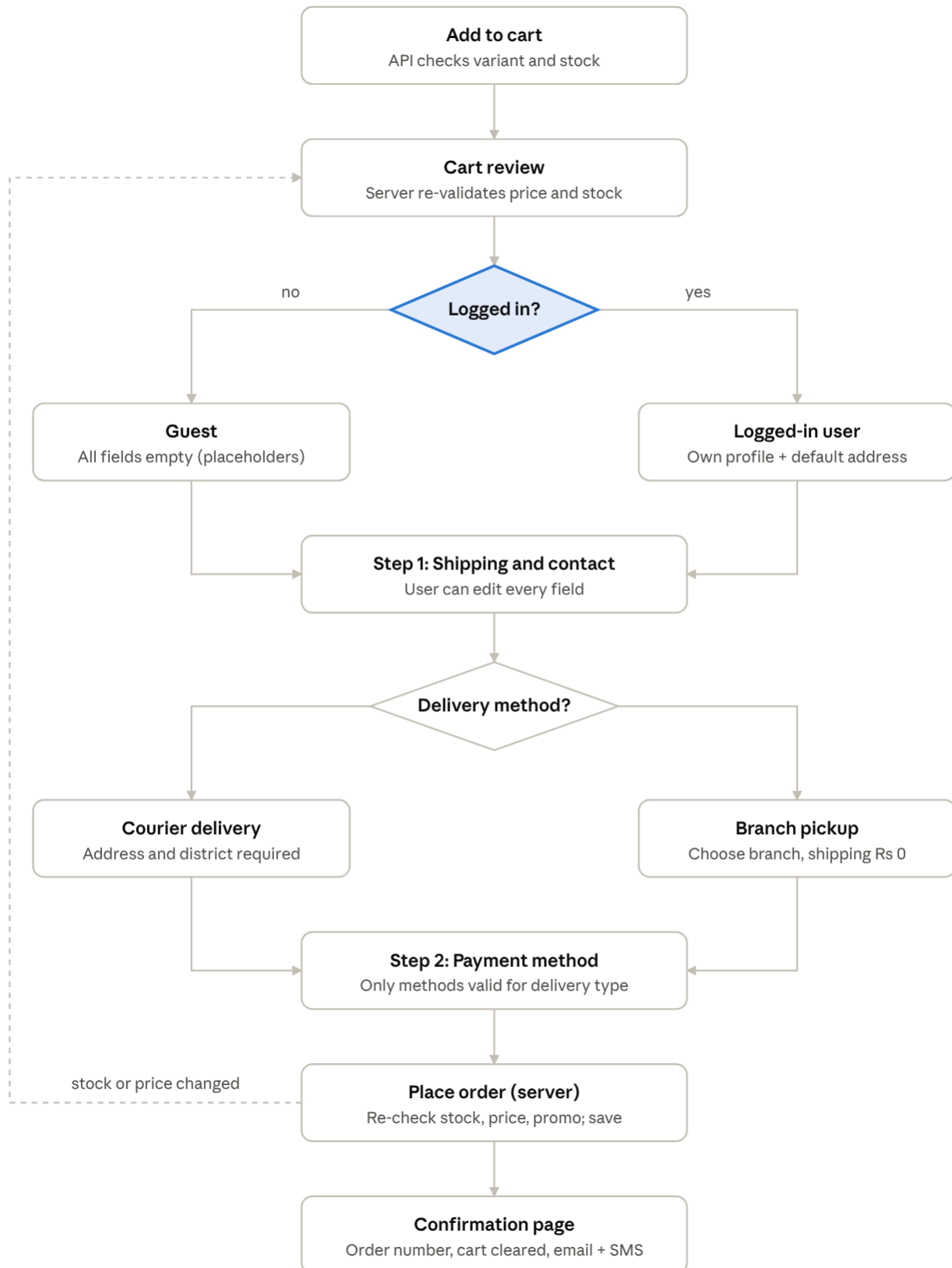
- Payment gateway integration details (only the contract with the gateway is described)
- Order fulfilment inside the ERP after the order is created (picking, courier handover)
- Returns and refunds

Goals

1. No hardcoded or dummy customer data anywhere in checkout.
2. Logged-in users see their own saved details prefilled and can edit them.
3. Guests see empty fields and can check out without an account.
4. Every price, discount and total is recalculated on the server before the order is saved.

3. End-to-end flow

Guests start empty; logged-in users see only their own details



checkout flow · 8 steps, 2 decisions

The login check decides what the form starts with: guests get empty fields, logged-in users get only their own saved data. If stock or price changed when the order is placed, the user is sent back to the cart. Card payments add a gateway step before confirmation (section 6).

4. Step-by-step requirements

4.1 Add to cart (Shop / product page)

1. User picks a variant (colour, storage) and clicks the cart icon or "Add to cart".
2. Frontend calls `POST /api/cart/items` with `productId`, `variantId`, `quantity`.
3. Server checks the variant exists, is active and has stock. If not, return an error and show a toast; do not add.
4. Cart badge in the header updates with the total item count from the API response (not a local counter).
5. Cart belongs to the user if logged in; for guests it is tied to a `cartId` stored in a cookie. On login, the guest cart is merged into the user cart.

4.2 Cart page

1. Shows each line: image, name, variant (e.g. Charcoal), unit price, quantity, line total.
2. User can change quantity (1 to available stock, max 5 per item) or remove a line. Every change goes through the API.
3. "Proceed to checkout" is disabled when the cart is empty.
4. Before moving on, the server re-validates stock and prices. Any changed price or out-of-stock line is shown to the user with a message.

4.3 Checkout step 1: Shipping & Contact Details

Prefill rules (this fixes the current bug):

User state	What the form shows
Guest (not logged in)	All fields empty. Placeholders only (e.g. "077 123 4567"), shown in grey, never as values.
Logged in, no saved address	Full Name, Phone and Email from the user profile. Address fields empty.
Logged in, has saved addresses	Profile contact details + the address marked as default. A dropdown lets the user pick another saved address or "Use a new address".

User state	What the form shows
Admin / staff account	Treat exactly like any other user. If staff are placing an order for a walk-in customer, all fields start empty.

- Placeholder text must use the HTML `placeholder` attribute, never `value` or `defaultValue`.
- Remove every hardcoded default from the form's initial state (e.g. `useState({ name: "Mobixa Admin", ... })` must become empty strings).
- Prefill data comes from `GET /api/me/checkout-profile`. While it loads, show skeleton fields, not dummy text.
- Delivery Instructions starts empty for everyone.
- A "Save this address for next time" checkbox appears for logged-in users when a new address is entered.

Delivery method toggle:

- Islandwide Courier Delivery (1-3 days)*: shows address fields (Line 1, Line 2, City, District) and Delivery Instructions. Address fields are required.
- Branch Pickup (Express Collection)*: hides address fields and shows a required "Select branch" dropdown loaded from `GET /api/stores`. Show branch address and opening hours under the dropdown. Shipping fee is always 0.
- Switching method keeps the contact details (name, phone, email) the user already typed.

On "Continue": validate all required fields (see section 8). Show errors inline under each field. Save the step data to the checkout session so a page refresh does not lose it.

4.4 Checkout step 2: Payment method

- Methods: Card (via payment gateway), Cash on Delivery (courier only, up to a configurable limit), Pay at Branch (pickup only), Bank Transfer.
- Only methods valid for the chosen delivery method and order total are shown.
- Card details are entered on the gateway's hosted page or iframe. The Mobia server never receives or stores full card numbers.

4.5 Place order and confirmation

- "Complete & Place Order" is enabled only when steps 1 and 2 are valid.
- The button disables on click and shows a spinner to stop double orders.

3. On success, redirect to `/order/confirmation/:orderNumber` showing order number, items, total, delivery method, address or branch, and expected delivery date.
4. Cart is cleared and the header badge returns to 0.
5. A confirmation email and SMS are sent to the email and phone entered in step 1.

5. Order summary, pricing and promo codes

The server is the only source of truth for money; the frontend only displays what the API returns.

Order Summary panel

- Item count badge (e.g. "1 Item") = number of distinct lines; also show total quantity per line ("Qty: 2").
- Each line shows the product image from the database (not a placeholder), name, variant, quantity and line total.
- Line total = unit price × quantity. Example from the screenshot: Google 10A × 2 at Rs 189,900.00 must show a line total of Rs 379,800.00, not the unit price.

Totals

Row	Rule
Subtotal	Sum of all line totals
Discount	From a valid promo code or product sale price; shown as a negative amount
Shipping	Courier: from shipping rules (e.g. free above a set amount, else flat fee by district). Pickup: always Rs 0
Tax & Duty	"Included" when prices are tax-inclusive; otherwise calculated
Total Payable	Subtotal – Discount + Shipping + Tax

All amounts are stored as integers in cents (or as DECIMAL(12,2)), never as floats. Display format: `Rs 379,800.00`.

Promo codes

1. User enters a code and clicks Apply → `POST /api/checkout/promo` with the code.
2. Server checks: code exists, active, within date range, usage limit not reached, per-user limit, minimum order value, eligible products or categories.

3. Valid → return new totals; show the code as a removable chip and the discount row.
4. Invalid → show the server's reason under the input (e.g. "This code has expired"). Totals do not change.
5. Only one promo code per order. Codes are case-insensitive.
6. The code is validated again when the order is placed.

Recalculation triggers: quantity change, delivery method change, district change (shipping fee), promo applied or removed. Each trigger calls the API and replaces the totals shown.

6. Placing the order

An order is created once, inside a single database transaction, and only after the server has re-checked stock, prices and the promo code.

Server steps for `POST /api/orders`

1. Read the `Idempotency-Key` header (a UUID the frontend generates once per checkout). If an order already exists for that key, return it instead of creating a new one.
2. Load the cart from the database. Ignore any prices sent by the frontend.
3. Re-validate each line: product active, variant in stock, quantity allowed.
4. Recalculate subtotal, discount, shipping, tax and total.
5. Validate shipping/contact data again (same rules as section 8).
6. In one transaction: create the order and order items (with a copy of name, variant and unit price at time of purchase), reserve or deduct stock, record promo usage, clear the cart.
7. Set status by payment method:
 - Cash on Delivery / Pay at Branch / Bank Transfer → `PENDING_CONFIRMATION`
 - Card → `AWAITING_PAYMENT`, then return the gateway redirect URL
8. Return `orderNumber` (e.g. `MOB-2026-000123`) and the next action.

Card payments

- Stock is reserved for 15 minutes while status is `AWAITING_PAYMENT`.
- The gateway calls a webhook `POST /api/payments/webhook`. Verify the signature, then set the order to `PAID` or `PAYMENT_FAILED`.
- Never mark an order paid based only on the browser redirect.
- If payment fails or the reservation expires, release the stock and let the user retry from the confirmation page.

Order statuses: PENDING_CONFIRMATION → CONFIRMED → PROCESSING → SHIPPED / READY_FOR_PICKUP → DELIVERED / COLLECTED . Side paths: AWAITING_PAYMENT , PAID , PAYMENT_FAILED , CANCELLED .

6.1 Admin Orders & Billing (currently broken)

Every order placed on the website must appear in the admin panel's Orders and Billing sections immediately, with all its products. Today, orders placed from the shop do not show up there.

Likely causes to check first

- The website checkout saves orders to a different table or collection (or only to local state / localStorage) than the one the admin Orders page reads.
- `order_items` are not saved, or not linked by `order_id` , so the admin sees an order with no products.
- The admin list filters by status, branch or source (e.g. only `POS` orders), which hides `WEB` orders.
- The Place Order button shows the confirmation page without calling `POST /api/orders` at all.
- No invoice/bill record is created when a web order is placed, so Billing stays empty.

Required behaviour

1. POS (in-store) orders and website orders use the same `orders` and `order_items` tables. Add a `source` column: `WEB` or `POS` .
2. When `POST /api/orders` succeeds, in the same transaction: save the order, save every order item (product, variant, name, unit price, quantity, line total), deduct or reserve stock, and create an invoice record.
3. Invoice number is generated by the server (e.g. `INV-2026-000123`) and linked to the order. Invoice status follows payment: `UNPAID` → `PAID` (card webhook, cash collected on delivery, or payment at branch) or `VOID` if the order is cancelled.
4. Admin Orders list shows web orders by default, newest first, with columns: order number, date, customer, source, delivery method, branch, items count, total, payment status, order status.
5. Admin order detail page shows every product line, customer contact, shipping address or pickup branch, promo code, totals and payment history.
6. Admin Billing page lists invoices with order number, customer, amount, payment method and status, and can print or download the invoice as PDF.
7. Admin can move the order through its statuses (section 6) and mark cash/branch payments as received. Each change is logged with the admin user and time.

8. New web orders show a notification or badge count in the admin panel.

Admin API endpoints

Method	Endpoint	Purpose
GET	/api/admin/orders? source=&status=&from=&to=	List orders with filters and paging
GET	/api/admin/orders/:id	Order detail with items, customer, payments
PATCH	/api/admin/orders/:id/status	Change order status
GET	/api/admin/invoices	List invoices (Billing page)
GET	/api/admin/invoices/:id/pdf	Printable invoice
POST	/api/admin/invoices/:id/payments	Record cash or branch payment

All admin endpoints require an admin or staff role.

7. Data model and API

Table and endpoint names below are suggestions; match them to the existing Mobia backend where equivalents already exist.

Main tables

Table	Key fields
users	id, full_name, email, phone, role
addresses	id, user_id, label, line1, line2, city, district, is_default
carts	id, user_id (nullable), guest_token, updated_at
cart_items	id, cart_id, product_id, variant_id, quantity
stores	id, name, address, city, district, phone, opening_hours, pickup_enabled
promo_codes	id, code, type (percent/fixed), value, min_order, starts_at, ends_at, usage_limit, per_user_limit

Table	Key fields
orders	id, order_number, user_id (nullable), status, delivery_method, store_id, contact_name, contact_phone, contact_email, ship_line1, ship_line2, ship_city, ship_district, notes, subtotal, discount, shipping_fee, tax, total, promo_code_id, payment_method, idempotency_key, created_at
order_items	id, order_id, product_id, variant_id, product_name, variant_name, unit_price, quantity, line_total
payments	id, order_id, gateway, gateway_ref, amount, status, raw_response, created_at

Orders store a copy of contact and address data, so later profile edits do not change past orders.

Endpoints

Method	Endpoint	Purpose
GET	/api/cart	Current cart with lines and totals
POST	/api/cart/items	Add item
PATCH	/api/cart/items/:id	Change quantity
DELETE	/api/cart/items/:id	Remove item
GET	/api/me/checkout-profile	Logged-in user's name, phone, email and saved addresses (401 for guests)
POST	/api/me/addresses	Save a new address
GET	/api/stores?pickup=true	Branches available for pickup
POST	/api/checkout/quote	Totals for a given delivery method, district and promo code
POST	/api/checkout/promo	Validate and apply promo code
DELETE	/api/checkout/promo	Remove promo code
POST	/api/orders	Place order (needs Idempotency-Key header)
GET	/api/orders/:orderNumber	Confirmation page data (owner or guest token only)
POST	/api/payments/webhook	Payment gateway callback

Example: GET /api/me/checkout-profile

```
{
  "fullName": "Nimal Perera",
  "phone": "+94771234567",
  "email": "nimal@example.com",
  "addresses": [
    { "id": 12, "label": "Home", "line1": "45 Galle Road", "line2": "",
      "city": "Dehiwala", "district": "Colombo", "isDefault": true }
  ]
}
```

Example: POST /api/orders request body

```
{
  "deliveryMethod": "COURIER",
  "contact": { "fullName": "Nimal Perera", "phone": "+94771234567", "email":
"nimal@example.com" },
  "shipping": { "addressId": 12 },
  "storeId": null,
  "notes": "",
  "paymentMethod": "CARD",
  "promoCode": "MOBI10",
  "saveAddress": false
}
```

For a new address, send `shipping` as `{ line1, line2, city, district }` instead of `addressId`. For pickup, send `deliveryMethod: "PICKUP"`, a `storeId` and no `shipping`. The body contains no prices.

7.1 Backend logic (procedures)

Each procedure below is the exact server-side logic for one endpoint. It is written as stack-neutral pseudocode; the developer implements it in the Mobia backend's language and ORM. Anything marked `TRANSACTION` must run as one database transaction: if any step fails, everything rolls back.

P1. `getCheckoutProfile` — `GET /api/me/checkout-profile`

```
PROCEDURE getCheckoutProfile(session)
  IF session.user IS NULL THEN
    RETURN 401 Unauthorized          -- frontend then shows empty fields
  user      = SELECT id, full_name, phone, email FROM users WHERE id =
session.user.id
  addresses = SELECT * FROM addresses WHERE user_id = user.id
              ORDER BY is_default DESC, id DESC
  RETURN 200 { fullName: user.full_name, phone: user.phone,
              email: user.email, addresses }
  -- NEVER return hardcoded or admin values; NEVER read another user's rows
END
```

P2. `addToCart` — `POST /api/cart/items`

```

PROCEDURE addToCart(session, productId, variantId, quantity)
  VALIDATE quantity BETWEEN 1 AND 5
  variant = SELECT * FROM product_variants
              WHERE id = variantId AND product_id = productId AND is_active = 1
  IF variant IS NULL THEN RETURN 404 "Product not available"
  cart = getOrCreateCart(session)      -- by user_id, or by guest_token
  cookie
  line = SELECT * FROM cart_items WHERE cart_id = cart.id AND variant_id =
variantId
  newQty = (line.quantity OR 0) + quantity
  IF newQty > variant.stock_available THEN RETURN 409 "Only {stock} left"
  UPSERT cart_items (cart_id, product_id, variant_id, quantity = newQty)
  RETURN 200 buildCartResponse(cart)   -- includes itemCount for header badge
END

```

P3. calculateTotals — used by cart, `POST /api/checkout/quote`, promo and placeOrder

```

PROCEDURE calculateTotals(cart, deliveryMethod, district, promoCode, userId)
  lines = SELECT ci.*, v.price, v.stock_available, p.name, v.name AS
variant_name
              FROM cart_items ci JOIN product_variants v JOIN products p
              WHERE ci.cart_id = cart.id
  errors = []
  FOR EACH line IN lines
    IF v.inactive OR line.quantity > line.stock_available THEN
      errors.ADD({ line, reason: "OUT_OF_STOCK" })
    line.unitPrice = line.price      -- ALWAYS from DB, never from
request
    line.lineTotal = line.unitPrice * line.quantity
  subtotal = SUM(lines.lineTotal)

  discount = 0
  IF promoCode IS NOT NULL THEN
    promo = validatePromo(promoCode, subtotal, lines, userId)  -- see P4
    IF promo.valid THEN discount = promo.amount
    ELSE errors.ADD({ reason: promo.reason })

  IF deliveryMethod = "PICKUP" THEN shipping = 0
  ELSE shipping = getShippingFee(district, subtotal - discount)

  tax    = 0      -- prices are tax-inclusive
  total = subtotal - discount + shipping + tax
  RETURN { lines, subtotal, discount, shipping, tax, total, errors }
END

```

P4. validatePromo

```

PROCEDURE validatePromo(code, subtotal, lines, userId)
  promo = SELECT * FROM promo_codes WHERE UPPER(code) = UPPER(:code) AND
is_active = 1
  IF promo IS NULL THEN RETURN invalid("Invalid
code")
  IF NOW() NOT BETWEEN starts_at AND ends_at THEN RETURN invalid("This code
has expired")
  IF promo.used_count >= promo.usage_limit THEN RETURN invalid("Usage limit
reached")
  IF userId AND COUNT(promo_usages WHERE promo_id AND user_id) >=
per_user_limit
THEN RETURN invalid("Already
used")
  IF subtotal < promo.min_order THEN RETURN invalid("Minimum
order Rs {min}")
  eligible = lines filtered by promo's products/categories (all if none set)
  base = SUM(eligible.lineTotal)
  amount = promo.type = "PERCENT" ? ROUND(base * promo.value / 100, 2) :
promo.value
  amount = MIN(amount, base, promo.max_discount OR amount)
  RETURN { valid: true, promo, amount }
END

```

P5. placeOrder — `POST /api/orders` (the main procedure; this is what must also feed admin Orders & Billing)

```

PROCEDURE placeOrder(session, idempotencyKey, body)
  -- 1. Duplicate protection
  existing = SELECT * FROM orders WHERE idempotency_key = idempotencyKey
  IF existing THEN RETURN 200 existing

  -- 2. Validate input (same rules as section 8)
  VALIDATE body.contact.fullName, phone (normalise to +947XXXXXXX), email
  IF body.deliveryMethod = "COURIER" THEN
    address = body.shipping.addressId
    ? SELECT * FROM addresses WHERE id = :id AND user_id =
session.user.id
    : VALIDATE body.shipping.{line1, city, district}
  ELSE IF body.deliveryMethod = "PICKUP" THEN
    store = SELECT * FROM stores WHERE id = body.storeId AND pickup_enabled
= 1
    IF store IS NULL THEN RETURN 422 "Select a valid branch"
  VALIDATE body.paymentMethod allowed for deliveryMethod
  IF errors THEN RETURN 422 { fieldErrors }

  cart = getCart(session)

```

```

IF cart IS EMPTY THEN RETURN 409 "Your cart is empty"

BEGIN TRANSACTION
  -- 3. Lock stock rows so two customers can't buy the last unit
  SELECT * FROM product_variants WHERE id IN (cart variant ids) FOR UPDATE

  -- 4. Recalculate everything on the server
  t = calculateTotals(cart, body.deliveryMethod, address.district,
                      body.promoCode, session.user?.id)
  IF t.errors NOT EMPTY THEN ROLLBACK; RETURN 409 { errors: t.errors }

  -- 5. Create order header (source = WEB so admin can filter)
  orderNumber = nextSequence("MOB-{YYYYY}-", 6 digits)
  status = body.paymentMethod = "CARD" ? "AWAITING_PAYMENT" :
"PENDING_CONFIRMATION"
  order = INSERT INTO orders (order_number, user_id, source = "WEB",
status,
                        delivery_method, store_id, contact_*, ship_* (copied, not
referenced),
                        notes, subtotal, discount, shipping_fee, tax, total,
                        promo_code_id, payment_method, idempotency_key, created_at =
NOW())

  -- 6. Create order lines (snapshot of name + price)
  FOR EACH line IN t.lines
    INSERT INTO order_items (order_id = order.id, product_id, variant_id,
                        product_name, variant_name, unit_price, quantity, line_total)

  -- 7. Stock
  FOR EACH line IN t.lines
    IF status = "AWAITING_PAYMENT" THEN
      UPDATE product_variants SET stock_reserved = stock_reserved + qty
      INSERT INTO stock_reservations (order_id, variant_id, qty,
                                expires_at = NOW() + 15 MINUTES)
    ELSE
      UPDATE product_variants SET stock_available = stock_available - qty
      INSERT INTO stock_movements (variant_id, qty = -qty, reason =
"WEB_ORDER", order_id)

  -- 8. Billing: invoice created with the order
  invoiceNumber = nextSequence("INV-{YYYYY}-", 6 digits)
  INSERT INTO invoices (invoice_number, order_id = order.id, customer_name,
                        amount = t.total, status = "UNPAID", issued_at = NOW())

  -- 9. Promo usage
  IF t.promo THEN

```

```

        UPDATE promo_codes SET used_count = used_count + 1 WHERE id =
t.promo.id
        INSERT INTO promo_usages (promo_id, user_id, order_id)

-- 10. Save new address if asked
IF body.saveAddress AND session.user THEN INSERT INTO addresses (...)

-- 11. Clear cart + audit log
DELETE FROM cart_items WHERE cart_id = cart.id
INSERT INTO order_status_history (order_id, status, changed_by =
"SYSTEM", at = NOW())
COMMIT

-- 12. After commit (outside transaction; failures here must not undo the
order)
QUEUE sendOrderEmail(order), sendOrderSms(order)
QUEUE notifyAdmins("New web order " + orderNumber)
IF body.paymentMethod = "CARD" THEN
    url = gateway.createPayment(orderNumber, t.total, returnUrl, webhookUrl)
    RETURN 201 { orderNumber, next: "REDIRECT", url }
RETURN 201 { orderNumber, next: "CONFIRMATION" }
END

```

P6. handlePaymentWebhook — POST /api/payments/webhook

```

PROCEDURE handlePaymentWebhook(request)
    IF NOT gateway.verifySignature(request) THEN RETURN 401
    order = SELECT * FROM orders WHERE order_number = request.orderRef FOR
UPDATE
    IF order.status <> "AWAITING_PAYMENT" THEN RETURN 200 -- already handled
    INSERT INTO payments (order_id, gateway, gateway_ref, amount, status,
raw_response)
    BEGIN TRANSACTION
        IF request.status = "SUCCESS" AND request.amount = order.total THEN
            UPDATE orders SET status = "PAID"
            UPDATE invoices SET status = "PAID", paid_at = NOW() WHERE order_id =
order.id
            convert stock_reservations -> stock_available - qty; delete
reservations
        ELSE
            UPDATE orders SET status = "PAYMENT_FAILED"
            release stock_reservations (stock_reserved - qty)
            INSERT INTO order_status_history (...)
        COMMIT
    RETURN 200
END

```

P7. releaseExpiredReservations — scheduled job, every 1 minute

```

PROCEDURE releaseExpiredReservations()
  FOR EACH r IN SELECT * FROM stock_reservations WHERE expires_at < NOW()
  BEGIN TRANSACTION
    UPDATE product_variants SET stock_reserved = stock_reserved - r.qty
    UPDATE orders SET status = "PAYMENT_FAILED" WHERE id = r.order_id
                                AND status = "AWAITING_PAYMENT"
    DELETE FROM stock_reservations WHERE id = r.id
  COMMIT
END

```

P8. Admin: listOrders / recordPayment / changeStatus

```

PROCEDURE adminListOrders(filters, page)
  REQUIRE role IN (ADMIN, STAFF)
  SELECT o.*, COUNT(oi.id) AS item_count, i.invoice_number, i.status AS
payment_status
  FROM orders o
  LEFT JOIN order_items oi ON oi.order_id = o.id
  LEFT JOIN invoices i    ON i.order_id = o.id
  WHERE (:source IS NULL OR o.source = :source)      -- default: ALL sources
    AND (:status IS NULL OR o.status = :status)
    AND o.created_at BETWEEN :from AND :to
  GROUP BY o.id ORDER BY o.created_at DESC LIMIT 25 OFFSET page*25
END

PROCEDURE adminRecordPayment(invoiceId, amount, method, adminId)
  REQUIRE role IN (ADMIN, STAFF)
  BEGIN TRANSACTION
    INSERT INTO payments (order_id, gateway = method, amount, status =
"SUCCESS")
    IF SUM(payments.amount) >= invoice.amount THEN
      UPDATE invoices SET status = "PAID", paid_at = NOW()
      INSERT INTO audit_log (admin_id, action = "PAYMENT_RECORDED", invoice_id)
    COMMIT
  END

PROCEDURE adminChangeStatus(orderId, newStatus, adminId)
  REQUIRE role IN (ADMIN, STAFF)
  IF transition(order.status -> newStatus) NOT ALLOWED THEN RETURN 422
  IF newStatus = "CANCELLED" THEN
    return stock to stock_available; UPDATE invoices SET status = "VOID"
  UPDATE orders SET status = newStatus

```

```

INSERT INTO order_status_history (order_id, status, changed_by = adminId,
at = NOW())
  QUEUE notifyCustomer(order, newStatus)
END

```

Extra tables needed by these procedures: `product_variants` (id, product_id, name, price, stock_available, stock_reserved, is_active), `invoices` (id, invoice_number, order_id, customer_name, amount, status, issued_at, paid_at), `stock_reservations`, `stock_movements`, `promo_usages`, `order_status_history`, `audit_log`.

8. Validation and edge cases

The same rules run in the browser (for fast feedback) and on the server (as the real check).

Field rules

Field	Required	Rule	Error message
Full Name	Yes	2–80 characters, letters, spaces, . ' -	Please enter your full name
Phone Number	Yes	Sri Lankan mobile: 07XXXXXXXX or +947XXXXXXXX; store as +947XXXXXXXX	Enter a valid mobile number (e.g. 0771234567)
Email	Yes	Valid email format, max 120 characters	Enter a valid email address
Address Line 1	Courier only	5–120 characters	Please enter your address
Address Line 2	No	Max 120 characters	—
City / Town	Courier only	2–60 characters	Please enter your city
District	Courier only	One of the 25 districts of Sri Lanka, from a fixed list; no default selected	Please select your district
Branch	Pickup only	Active store with pickup enabled	Please select a pickup branch
Delivery Instructions	No	Max 300 characters	—

Field	Required	Rule	Error message
Payment Method	Yes	Allowed for chosen delivery method and total	Please choose a payment method

Edge cases

Situation	Expected behaviour
Item goes out of stock during checkout	Block order, show which item, send user back to cart
Price changed since added to cart	Show old vs new price, ask user to confirm before placing
Promo code expires between Apply and Place Order	Remove discount, show message, update total, require re-confirm
User double-clicks Place Order	One order only (button disabled + Idempotency-Key)
Network error after Place Order	Retry with the same Idempotency-Key; never create a second order
User logs in mid-checkout	Merge guest cart; keep typed fields, offer saved addresses
Session expires	Keep cart; ask to log in again or continue as guest
Cart emptied in another tab	Redirect to cart with "Your cart is empty"
Payment gateway timeout	Order stays AWAITING_PAYMENT; webhook decides final state; show retry option
Direct visit to /checkout with empty cart	Redirect to /cart

Error display: field errors inline under the field in red; order-level errors in a banner above the Place Order button. Never show raw server errors or stack traces.

9. Security and non-functional requirements

- **No test data in production code.** Remove all dummy names, phones, emails and addresses from components, seed files loaded at runtime and form defaults. Test data lives only in test fixtures.

- **Auth:** `/api/me/*` endpoints require a valid session and return only the caller's own data. A guest can view an order only with the order number plus a signed guest token sent in the confirmation email.
- **Never trust the client:** prices, discounts, shipping fees, stock and totals come from the server.
- **Payments:** card data handled only by the gateway (PCI DSS). Verify webhook signatures. Log gateway responses without card data.
- **Personal data:** HTTPS only; do not log full phone numbers or addresses in application logs; follow Sri Lanka's Personal Data Protection Act No. 9 of 2022.
- **Rate limits:** promo endpoint max 10 attempts per user/IP per 10 minutes; order endpoint max 5 per minute per user/IP.
- **Performance:** checkout page interactive in under 2 seconds on 4G; each totals recalculation under 500 ms.
- **Mobile:** single-column layout below 768 px; Order Summary collapsible and shown above the Place Order button.
- **Accessibility:** every field has a label, errors are linked to fields (`aria-describedby`), full keyboard navigation.

10. Acceptance criteria

The work is done when every item below passes on staging.

- ☐ Searching the frontend code for "Mobixa Admin", "+94 77 123 4567", "admin@mobilehub.com" and "Tech Avenue" returns no results outside test fixtures
- ☐ Guest adds an item → checkout opens with every field empty (placeholders only)
- ☐ Logged-in user with no saved address → name, phone and email are theirs; address fields empty
- ☐ Logged-in user with a default address → that address is prefilled and can be changed or replaced
- ☐ Two different user accounts see only their own details at checkout
- ☐ Branch Pickup hides address fields, requires a branch and sets shipping to Rs 0
- ☐ Required-field errors appear inline and block Continue
- ☐ Changing quantity, district or delivery method updates totals from the API
- ☐ Valid promo code applies the discount; expired or invalid code shows the reason and changes nothing
- ☐ Editing prices in browser dev tools does not change the saved order total
- ☐ Double-clicking Place Order creates exactly one order
- ☐ Out-of-stock item at order time blocks the order with a clear message

- ☐ Card payment marked PAID only after a verified webhook
- ☐ After a successful order: confirmation page shows correct details, cart badge shows 0, email and SMS received
- ☐ Order record keeps the contact and address used, even after the user edits their profile

Admin Orders & Billing

- ☐ A web order placed as a guest appears in admin Orders within 5 seconds, with every product line and correct totals
- ☐ A web order placed by a logged-in user appears the same way and is linked to that user
- ☐ Each web order has an invoice in admin Billing with a unique invoice number
- ☐ Card order: invoice turns PAID only after the verified webhook
- ☐ Cash on delivery / pay at branch: admin can record the payment and the invoice turns PAID
- ☐ Stock count in admin drops by the ordered quantity
- ☐ Admin can filter orders by source (WEB / POS) and status
- ☐ Invoice PDF shows the same products, prices and total as the customer's confirmation page